

Padding Oracle Attack a AES

Pesenti Michele - 41488A

12 aprile 2026

Indice

1	Advanced Encryption Standard - AES	3
1.1	Creazione S-box e IS-box	3
1.2	Schedulazione delle Chiavi	4
1.3	Cifratura	5
1.4	Decifratura	6
1.5	Operazioni di Round	7
1.6	Attacchi Base	9
2	Cipher Block Chaining - CBC	10
2.1	Architettura della Modalità CBC	10
2.2	Cifratura e Decifratura	10
2.3	Gestione del Padding	11
2.4	Sicurezza e Criticità della Modalità CBC	11
2.5	CBC Applicato ad AES	12
3	Padding PKCS#7	13
3.1	Standard PKCS#7	13
3.2	Padding in CBC	14
4	Padding Oracle Attack	15
4.1	Oracolo	15
4.2	Meccanismo dell'Attacco	16
4.3	Logica dell'Attacco	16
4.4	Esempio Logica dell'Attacco	17
4.5	Contromisure	18

1 Advanced Encryption Standard - AES

Lo standard AES (Advanced Encryption Standard) è stato pubblicato dal National Institute of Standards and Technology (NIST) nel 2001. AES è il cifrario a blocchi simmetrico maggiormente utilizzato.

Il cifrario riceve come input un blocco di testo di dimensione pari a 16 byte, ovvero 128 bit. La lunghezza della chiave può essere di 16, 24 o 32 byte (128, 192 o 256 bit). L'algoritmo è indicato come AES-128, AES-192 o AES-256, a seconda della lunghezza della chiave.

Per semplicità spiegherò prendendo come base l'algoritmo che lavora con chiave da 128 bit, ma il ragionamento è identico anche con chiavi più grandi.

1.1 Creazione S-box e IS-box

In AES la S-box (Substitution box) è uno degli elementi centrali per garantire la sicurezza dell'algoritmo, poiché introduce non linearità nel processo di cifratura. Senza di essa, AES sarebbe sostanzialmente un sistema lineare e quindi facilmente attaccabile.

S-box è utilizzata sia nei round che nella schedulazione della chiave. Durante ogni round di cifratura AES applica l'operazione SubBytes, in cui ogni byte dello stato viene sostituito tramite la S-box. Questa sostituzione permette di rompere le relazioni lineari tra input e output, indurre confusione e quindi rendere inefficaci attacchi come quelli lineari e differenziali.

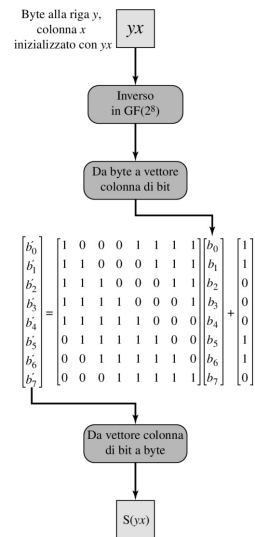
Inoltre la S-box viene utilizzata anche nella fase di espansione della chiave, in particolare nell'operazione chiamata SubWord. È applicata agli elementi della chiave per introdurre non linearità nella generazione delle sottochiavi ed evitare che sottochiavi diverse siano legate tra loro da relazioni semplici. Questo è fondamentale perché se la schedulazione della chiave fosse lineare un attaccante potrebbe sfruttarlo per risalire alla chiave originale.

La S-box di AES non è casuale ma è costruita matematicamente in questo modo:

1. Si inizializza la S-box con il valore dei byte in una sequenza crescente riga per riga. La prima riga conterrà $\{00\}, \{01\}, \dots, \{0F\}$; la seconda $\{10\} \{11\} \dots \{1F\}$ e così via fino a $\{F0\} \{F1\} \dots \{FF\}$. Il valore del byte nella riga Y è colonna X sarà $\{YX\}$
2. Si mappa ciascun byte nella S-box con il suo inverso moltiplicativo nel campo finito $GF(2^8)$. Ovvero trovare $a^{-1} \mid a \cdot a^{-1} = 1$. Nota: l'inverso di $\{00\}$ è $\{00\}$
3. Si applica la trasformazione ad ogni bit che costituisce il byte $(b_7, b_6, b_5, b_4, b_3, b_2, b_1, b_0)$ applicando $b'_i = b_i \oplus b_{i+4} \oplus b_{i+5} \oplus b_{i+6} \oplus b_{i+7} \oplus c_i$. Con c_i che è l'i-esimo bit della costante byte c, che vale $\{63\}$ ed è composto da $(c_7, c_6, c_5, c_4, c_3, c_2, c_1, c_0) = (01100011)$

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
	1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
	2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
	3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
	4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
	5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
	6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
	7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
	8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
	9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
	A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
	B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
	C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
	D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
	E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
	F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

(a) S-box



(b) Calcolo byte di S-box in posizione YX

La IS-box è l'inverso della S-box e viene usata esclusivamente nella fase di decifratura durante l'operazione InvSubBytes; serve ad annullare l'effetto della S-box applicata in cifratura, permettendo così di ricostruire il testo in

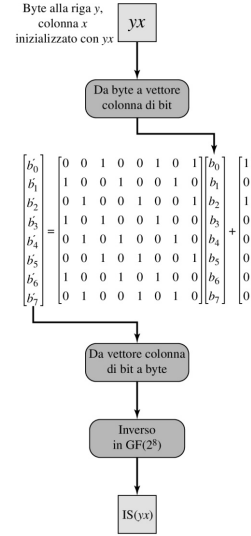
chiaro.

Si modificano i byte applicando l'inversa dell'equazione usata per la costruzione di S-box, quindi:

$b'_i = b_{(i+2) \bmod 8} \oplus b_{(i+5) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus d_i$. Con d_i che è l'i-esimo bit della costante byte d, che vale {05} ed è composto da $(d_7, d_6, d_5, d_4, d_3, d_2, d_1, d_0) = (00000101)$.

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
	1	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9	CB
	2	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3	4E
	3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25
	4	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6	92
	5	6C	70	48	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D	84
	6	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06
	7	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A	6B
	8	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6	73
	9	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF	6E
	A	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE	1B
	B	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A	F4
	C	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC	5F
	D	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C	EF
	E	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99	61
	F	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C	7D

(a) IS-box



(b) Calcolo byte di IS-box in posizione YX

1.2 Schedulazione delle Chiavi

L'algoritmo di espansione della chiave di AES richiede come input una chiave di 4 *word* (16 byte, 1 *word* equivale a 4 byte o 32 bit) e produce un vettore lineare di 44 *word* (176 byte o 1408 bit). Questo è sufficiente per fornire una chiave di round di 4 *word* per la trasformazione iniziale AddRoundKey e anche per ciascuno dei 10 round di cifratura.

La chiave viene copiata nelle prime quattro parole della chiave espansa. Il resto della chiave espansa viene riempito quattro parole alla volta. Ciascuna parola $w[i]$ dipende dalla parola immediatamente precedente, $w[i-1]$, e dalla parola memorizzata quattro posizioni indietro, $w[i-4]$. Se l'indice "i" non è multiplo di quattro allora si esegue un semplice XOR in cui $w[i] = w[i-1] \oplus w[i-4]$.

Per un word la cui posizione nel vettore w è un multiplo di 4, viene utilizzata una funzione più complessa chiamata funzione g , costituita dalle seguenti sottofunzioni:

1. RotWord: esegue uno scorrimento a sinistra circolare di un byte su un word; questo significa che un word in input $[B_0, B_1, B_2, B_3]$ viene trasformato in $[B_1, B_2, B_3, B_0]$
2. SubWord esegue una sostituzione di ciascun byte del word in input, utilizzando la S-Box
3. Il risultato delle operazioni precedenti è sottoposto a un'operazione XOR con una costante di round detta Rcon[j]

La costante di round è una parola in cui i tre byte più a destra sono sempre 0. Ciò significa che effettuare lo XOR di un word con la costante Rcon corrisponde a fare lo XOR solo con il byte più a sinistra della parola.

La costante di round è differente per ciascun round ed è definita come

$Rcon[j] = (RC[j], 0, 0, 0)$ con $RC[j] = 2 \cdot RC[j-1]$; e con la moltiplicazione definita sul campo $GF(2^8)$.

I valori di $RC[j]$ in esadecimale sono:

j	1	2	3	4	5	6	7	8	9	10
RC[j]	01	02	04	08	10	20	40	80	1B	36

Tabella 1: Valori di $RC[j]$ per j per AES-128

L'algoritmo di espansione della chiave è stato progettato per resistere agli attacchi noti di analisi crittografica. L'inclusione di una costante di round che dipende dallo specifico round considerato elimina la simmetria tra le modalità con cui le chiavi di round vengono generate. I criteri specifici utilizzati sono:

- La conoscenza di una parte della chiave di cifratura non consente il calcolo di molti bit della chiave di round
- La trasformazione è invertibile; ovvero, la conoscenza di qualsiasi Nk parole consecutive della chiave espansa, consente la ricostruzione dell'intera chiave espansa ($Nk =$ dimensione della chiave in parole)
- Velocità elevata su un'ampia gamma di processori
- Utilizzo di costanti di round per eliminare le simmetrie
- Diffusione delle differenze delle chiavi di cifratura nelle chiavi di round; ovvero, ciascun bit della chiave influisce su molti bit di round
- Non linearità sufficiente ad impedire la determinazione completa delle differenze tra chiavi di round solo a partire dalla conoscenza delle differenze tra due intere chiavi di cifratura

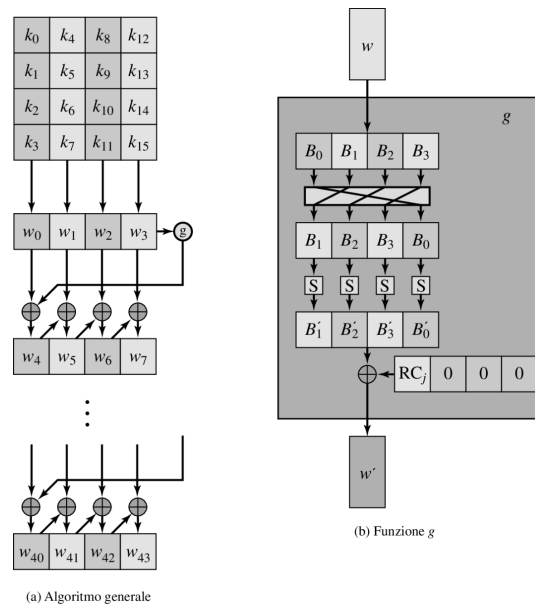


Figura 3: Schedulazione delle chiavi

1.3 Cifratura

L'input degli algoritmi di cifratura e di decifratura è un singolo blocco a 128 bit, rappresentato come una matrice quadrata di dimensione 4×4 byte. Questo blocco viene copiato nella matrice di stato, poi modificata in ogni round di cifratura o decifratura. Dopo il round finale, la matrice di Stato viene copiata in una matrice di output.

La chiave è rappresentata come una matrice quadrata di byte. Questa chiave viene poi espansa in un vettore di parole per la schedulazione delle chiavi. Ogni parola è di quattro byte e la chiave espansa completa è di 44 parole per chiavi a 128 bit.

Il cifrario è composto da N round e dipende dalla lunghezza della chiave: 10 round per una chiave da 16 byte, 12 round per una chiave da 24 byte e 14 round per una chiave da 32 byte.

È presente una singola trasformazione iniziale: AddRoundKey, considerato come round 0. Gli altri $N-1$ round consistono in quattro funzioni di trasformazione: SubBytes, ShiftRows, MixColumns e AddRoundKey; nel round finale, sono invece applicate solo tre trasformazioni.

Ogni trasformazione riceve come input una o più matrici 4×4 e produce come output una matrice 4×4 . Inoltre in ogni round è utilizzata una chiave di dimensioni 4×4 byte.

La trasformazione SubBytes induce confusione, che serve a rendere complessa la relazione tra chiave e testo cifrato; mentre ShiftRows e MixColumns inducono diffusione, il cui obiettivo è far sì che ogni bit del testo in chiaro influenzi più bit del testo cifrato.

L'omissione della trasformazione MixColumns nell'ultimo round di AES è una scelta progettuale mirata all'ottimizzazione e alla simmetria dell'algoritmo. Poiché la diffusione dei bit è già stata garantita dai round precedenti, un'ulteriore applicazione di MixColumns risulterebbe ridondante sotto il profilo della sicurezza, non apportando un incremento significativo della confusione o della diffusione.

Dal punto di vista strutturale, l'assenza di questa operazione nell'ultimo passaggio, permette di mantenere una corrispondenza più stretta tra il processo di cifratura e quello di decifratura. Se MixColumns fosse presente, la decifratura richiederebbe una trasformazione inversa aggiuntiva prima del primo AddRoundKey, complicando inutilmente l'implementazione sia hardware che software senza alcun guadagno effettivo in termini di robustezza crittografica.

1.4 Decifratura

La decifratura viene eseguita applicando lo stesso numero di round e lo stesso ordine della cifratura; quindi un primo round 0 con solo un AddRoundKey, seguito da N-1 round con 4 trasformazioni ed infine l'ultimo round con 3 trasformazioni.

A differenza però della cifratura, nei round di decifratura l'ordine delle trasformazioni è differente; si applica InvShiftRows, seguita da InvSubBytes, poi AddRoundKey ed infine InvMixColumns.

La trasformazione InvSubBytes, utilizza la IS-box, ovvero l'inversa della S-box. Utilizza lo stesso criterio di selezione degli indici come la trasformazione sorella e sostituisce il byte della matrice di stato con quello ricavato dalla IS-box.

La trasformazione inversa InvShiftRows esegue gli scorrimenti circolari nella direzione opposta per ciascuna delle ultime tre righe; quindi applica uno scorrimento circolare a destra di 1 byte per la seconda riga, poi 2 byte per la terza e 3 byte per la quarta.

La trasformazione InvMixColumns ha lo stesso funzionamento della sua trasformazione usata in cifratura, ma utilizza una matrice diversa:

$$\begin{pmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{pmatrix}$$

La moltiplicazione tra matrici:

$$\begin{pmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{pmatrix} \cdot \begin{pmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{pmatrix} = \begin{pmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{pmatrix}$$

Cioè è possibile perché, moltiplicando le due matrici utilizzate per la moltiplicazione con le matrici di stato in cifratura e decifratura, si ottiene una matrice d'identità.

$$\begin{pmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{pmatrix} \cdot \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \cdot \begin{pmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{pmatrix} = \begin{pmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{pmatrix}$$

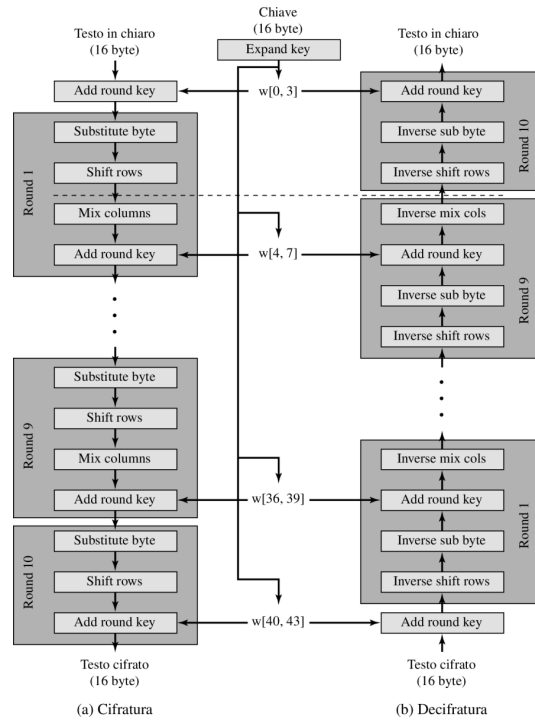


Figura 4: Struttura di cifratura e decifratura

1.5 Operazioni di Round

In ogni round, eccetto l'ultimo, vengono eseguite quattro trasformazioni: SubBytes, ShiftRows, MixColumns e AddRoundKey. La trasformazione SubBytes, è una semplice tabella di ricerca. AES definisce una matrice di 16x16 byte, chiamata S-box, che contiene una permutazione di tutti i possibili 256 valori a 8 bit.

Ogni singolo byte dello Stato viene mappato in un nuovo byte utilizzando i quattro bit più a sinistra del byte come valore di riga e i 4 bit più a destra come valore di colonna. Questi valori di riga e colonna sono utilizzati come indici nella S-box per la selezione di un valore univoco di output a 8 bit.

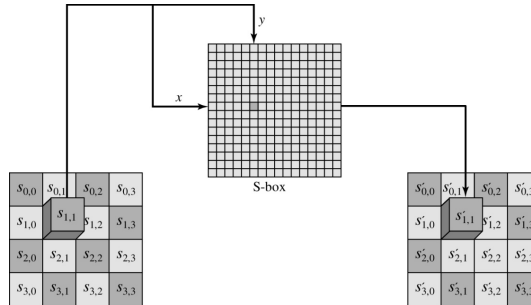


Figura 5: Trasformazione SubBytes

La trasformazione ShiftRows applica un semplice rotazione. La prima riga dello Stato non viene modificata, la seconda riga subisce uno scorrimento circolare a sinistra di 1 byte, la terza riga di 2 byte ed infine la quarta di 3 byte.

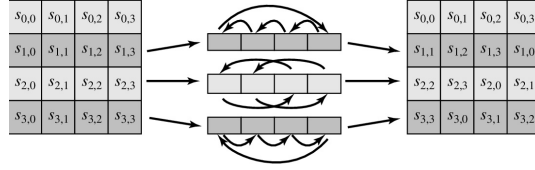


Figura 6: Trasformazione ShiftRows

La trasformazione MixColumns, opera su ogni colonna singolarmente. Ciascun byte di una colonna viene mappato in un nuovo valore che è una funzione di tutti e quattro i byte presenti in quella colonna. La trasformazione può essere definita dalla moltiplicazione di matrici sulla matrice di Stato:

$$\begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \cdot \begin{pmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{pmatrix} = \begin{pmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{pmatrix}$$

Da cui si può dedurre che il valore di ogni singolo byte per la propria colonna può essere definito tramite queste equazioni:

$$\begin{aligned} s'_{0,j} &= (2 \cdot s_{0,j}) \oplus (3 \cdot s_{1,j}) \oplus s_{2,j} \oplus s_{3,j} \\ s'_{1,j} &= s_{0,j} \oplus (2 \cdot s_{1,j}) \oplus (3 \cdot s_{2,j}) \oplus s_{3,j} \\ s'_{2,j} &= s_{0,j} \oplus s_{1,j} \oplus (2 \cdot s_{2,j}) \oplus (3 \cdot s_{3,j}) \\ s'_{3,j} &= (3 \cdot s_{0,j}) \oplus s_{1,j} \oplus s_{2,j} \oplus (2 \cdot s_{3,j}) \end{aligned}$$

Ciascun elemento nella matrice prodotto è la somma dei prodotti degli elementi di una riga e di una colonna.

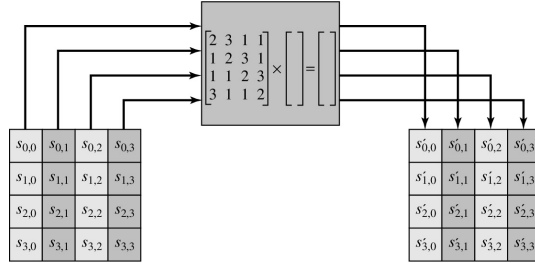


Figura 7: Trasformazione MixColumns

Nella trasformazione AddRoundKey i 128 bit della matrice di Stato vengono sottoposti a uno XOR bit-a-bit con i 128 bit della chiave di round.

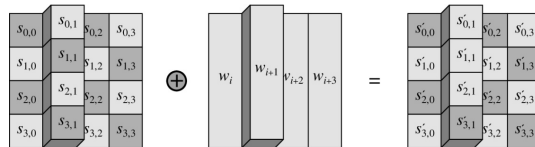


Figura 8: Trasformazione AddRoundKey

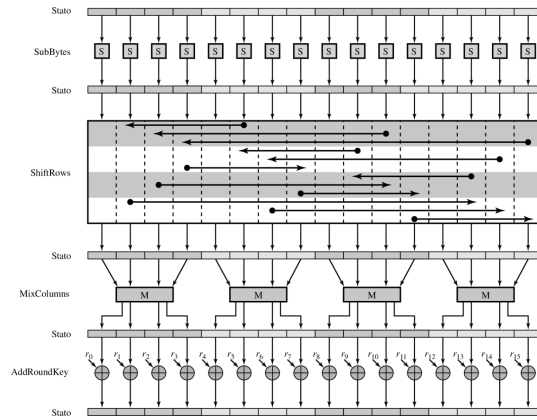


Figura 9: Trasformazioni in un round

1.6 Attacchi Base

L'algoritmo AES è considerato tutt'oggi sicuro dal punto di vista crittografico; non esistono attacchi pratici in grado di comprometterlo direttamente senza conoscere la chiave segreta. Tuttavia, ciò non implica che i sistemi che utilizzano AES siano automaticamente sicuri. Nella pratica, infatti, le vulnerabilità emergono quasi sempre dal modo in cui l'algoritmo viene implementato o utilizzato.

Una prima categoria di attacchi è rappresentata dai side-channel attacks, che non colpiscono la struttura matematica di AES ma la sua esecuzione su dispositivi reali. Tecniche come il timing attack, la power analysis o gli attacchi alla cache sfruttano informazioni collaterali, come i tempi di esecuzione o il consumo energetico, per dedurre informazioni sulla chiave segreta. Questi attacchi sono particolarmente efficaci in ambienti embedded o su hardware non protetto. Un'altra classe rilevante riguarda gli errori nell'uso delle modalità operative. AES non viene utilizzato direttamente sui dati, ma attraverso modalità come CBC, CTR, GCM ed ECB. Scelte errate, come l'impiego della modalità ECB, che non nasconde i pattern dei dati, possono compromettere la sicurezza anche se l'algoritmo sottostante è robusto. Inoltre, modalità come CBC risultano vulnerabili alla manipolazione se non accompagnate da meccanismi di autenticazione. In questi casi, l'attacco non sfrutta una debolezza intrinseca di AES, ma una cattiva progettazione del sistema crittografico.

Esistono poi attacchi più teorici, come quelli di tipo related-key o il biclique attack, che riducono marginalmente la complessità computazionale rispetto alla ricerca esaustiva della chiave. Tuttavia, tali approcci non sono considerati pratici e non rappresentano una minaccia concreta nei contesti reali.

È importante sottolineare che AES, considerato isolatamente, non è progettato per garantire sicurezza rispetto ad attacchi di tipo Chosen Ciphertext (IND-CCA). Questa proprietà dipende infatti dalla modalità operativa e dalla presenza di meccanismi di autenticazione. In particolare, alcune implementazioni basate su modalità CBC con padding possono introdurre vulnerabilità se il sistema fornisce informazioni durante la fase di decifratura.

Tra gli attacchi che sfruttano questo comportamento vi è il Padding Oracle Attack. Esso si applica a sistemi che, durante la decifratura, rivelano se il padding è corretto o meno. In presenza di tale "oracolo", un attaccante può inviare varianti modificate del testo cifrato e, analizzando le risposte del sistema, ricostruire progressivamente il testo in chiaro senza conoscere la chiave segreta.

2 Cipher Block Chaining - CBC

Le modalità operative dei cifrari a blocchi nascono dall'esigenza di estendere l'utilizzo di algoritmi che operano su blocchi di dimensione fissa, alla cifratura di messaggi di lunghezza arbitraria. In assenza di una modalità operativa, il cifrario agirebbe indipendentemente su ciascun blocco, esponendo il sistema a vulnerabilità significative, come nel caso della modalità ECB, che non garantisce adeguata sicurezza.

Tra le diverse modalità esistenti la Cipher Block Chaining (CBC) rappresenta una delle più diffuse. Essa introduce una dipendenza tra i blocchi di testo in chiaro, combinando ciascun blocco con il risultato della cifratura del blocco precedente prima di applicare l'algoritmo di cifratura. Questo meccanismo consente di migliorare la sicurezza rispetto a modalità più semplici eliminando la corrispondenza diretta tra blocchi uguali di input e output.

L'utilizzo della modalità CBC richiede inoltre l'impiego di un vettore di inizializzazione (Initialization Vector, IV), che garantisce la variabilità del primo blocco cifrato anche in presenza di input identici. Tuttavia, come verrà discusso più avanti, la sicurezza della modalità dipende fortemente dalla corretta gestione di tali parametri e dalle modalità con cui viene implementata.

Infine, poiché AES opera su blocchi di lunghezza fissa, risulta necessario adottare un meccanismo di padding per gestire messaggi la cui lunghezza non è multipla della dimensione del blocco. Tale aspetto, apparentemente secondario, riveste in realtà un ruolo cruciale nella sicurezza complessiva del sistema e sarà approfondito nel capitolo successivo.

2.1 Architettura della Modalità CBC

La modalità operativa Cipher Block Chaining (CBC) è progettata per migliorare la sicurezza dei cifrari a blocchi introducendo una dipendenza tra i blocchi di testo. In questa architettura, l'input della funzione di cifratura o decifratura per ciascun blocco non consente alcuna relazione fissa con il blocco di testo originale; di conseguenza, non vengono esposti pattern ripetuti di b bit, eliminando la vulnerabilità presente nella modalità ECB.

Il cuore dell'architettura risiede nel processo di concatenamento; ogni blocco di testo viene combinato con il blocco precedente di testo cifrato (o decifrato per la decifratura) prima di essere sottoposto all'algoritmo di cifratura (o decifratura). Questo processo è realizzato attraverso un'operazione XOR che garantisce che anche blocchi di testo identici producano output differenti.

Poiché il primo blocco del messaggio non dispone di un blocco precedente, l'architettura prevede l'utilizzo di un Vettore di Inizializzazione (IV), un blocco di dati che funge da "blocco zero" per avviare la catena. L'IV assicura che la cifratura sia probabilistica, ovvero che lo stesso messaggio cifrato, o decifrato, più volte produca testo cifrato diversi.

Infine, l'architettura CBC impone un vincolo strutturale sulla lunghezza dei dati; poiché l'algoritmo lavora su unità fisse di b bit, è necessario che l'ultimo blocco venga sottoposto a un'operazione di padding qualora si tratti di un blocco parziale, garantendo così che l'intera sequenza sia allineata alla dimensione richiesta dal cifrario.

2.2 Cifratura e Decifratura

Come già anticipato precedentemente, nella cifratura, ogni blocco di testo in chiaro viene combinato con il blocco cifrato precedente attraverso un'operazione XOR prima di essere sottoposto all'algoritmo di cifratura.

Nel caso iniziale, ovvero quando non esiste nessun blocco cifrato precedente, è utilizzato un Vettore di Inizializzazione (IV) che funge da blocco zero.

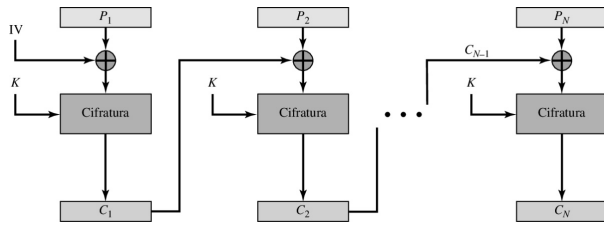
Si può formalizzare il processo di cifratura con le seguenti equazioni:

$$\begin{aligned}C_0 &= E(K, [P_0 \oplus IV]) \\C_j &= E(K, [P_j \oplus C_{j-1}]) \quad \text{per } j > 0\end{aligned}$$

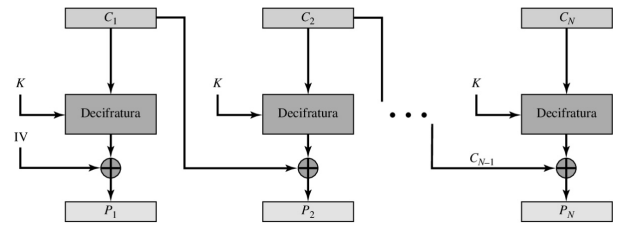
Per quanto riguarda la decifratura il processo è simile, ma con l'ordine delle operazioni invertito. Ogni blocco di testo cifrato viene decifrato e poi combinato con il blocco cifrato precedente (o IV per il primo blocco) per recuperare il testo in chiaro originale.

Si può formalizzare il processo di decifratura con le seguenti equazioni:

$$\begin{aligned}P_0 &= D(K, C_0) \oplus IV \\P_j &= D(K, C_j) \oplus C_{j-1} \quad \text{per } j > 0\end{aligned}$$



(a) Cifratura CBC



(b) Decifratura CBC

2.3 Gestione del Padding

Poiché la modalità CBC concatena blocchi di dimensione fissa, è raro che la lunghezza del messaggio originale sia un multiplo esatto di tale misura. La gestione del padding non è solo una necessità formale, ma un'operazione che avviene in due fasi distinte:

- **Cifratura:** il mittente calcola quanti byte mancano per completare l'ultimo blocco. Se il blocco non è completo viene riempito con byte di padding necessari per raggiungere la grandezza richiesta dall'algoritmo di cifratura. Nota: secondo lo standard PKCS#7, qualora il messaggio originale coincida con un multiplo del blocco, verrà aggiunto un intero blocco di padding extra. Questo aspetto verrà approfondito nella sezione dedicata allo standard
- **Decifratura:** una volta che l'algoritmo ha decifrato l'ultimo blocco e lo ha messo in XOR con il penultimo blocco cifrato (C_{j-1}), il sistema ottiene il testo in chiaro finale

2.4 Sicurezza e Criticità della Modalità CBC

La sicurezza della modalità Cipher Block Chaining (CBC) non dipende solo dalla robustezza del cifrario a blocchi sottostante, ma richiede condizioni rigorose sulla gestione dei parametri e sull'integrità dei dati.

Il vettore IV ha lo scopo di garantire che la cifratura sia probabilistica, ovvero, messaggi identici cifrati con la stessa chiave devono produrre testi cifrati differenti.

Perché il sistema sia sicuro l'IV deve essere non predicibile; se un avversario potesse conoscere in anticipo l'IV che verrà usato per il prossimo messaggio, potrebbe costruire un attacco per verificare ipotesi sul contenuto del testo in chiaro.

In conformità allo standard NIST SP 800-38A, i due requisiti fondamentali che l'IV deve soddisfare sono l'univocità (per evitare il riutilizzo con la medesima chiave) e l'imprevedibilità (garantita attraverso un generatore di numeri casuali crittograficamente sicuro o cifrando un nonce come un contatore o un timestamp mai utilizzato in precedenza).

Un limite fondamentale del CBC è che, pur garantendo la riservatezza, non garantisce l'integrità dei dati. Il sistema è intrinsecamente malleabile. Dall'equazione di decifratura:

$$P_j = D(K, C_j) \oplus C_{j-1}$$

si evince che il testo in chiaro P_j è legato linearmente al blocco cifrato precedente C_{j-1} .

$$D(K, C_j) = D(K, E(K, C_{j-1} \oplus P_j))$$

$$D(K, C_j) = C_{j-1} \oplus P_j$$

$$C_{j-1} \oplus D(K, C_j) = C_{j-1} \oplus C_{j-1} \oplus P_j = P_j$$

Questo implica che:

- se un attaccante altera un bit nel blocco C_{j-1} , il corrispondente bit in P_j verrà invertito dopo la decifratura. Tuttavia, la modifica di C_{j-1} corromperà completamente il blocco P_{j-1} (poiché C_{j-1} è anche l'input della funzione di decifratura per quel blocco), causando quella che viene chiamata propagazione dell'errore
- se un attaccante modifica un bit nel blocco C_j , il blocco P_j sarà completamente corrotto, ma i blocchi successivi P_{j+1} , P_{j+2} , ecc. rimarranno intatti. Questo è dovuto al fatto che ogni blocco di testo in chiaro dipende solo dal blocco cifrato immediatamente precedente e non da quelli successivi

Per una sicurezza completa contro modifiche attive, la modalità CBC dovrebbe essere sempre accompagnata da un meccanismo di autenticazione (come un HMAC).

2.5 CBC Applicato ad AES

L'applicazione dell'algoritmo AES all'interno della modalità CBC trasforma un cifrario a blocchi deterministico in un sistema di cifratura a catena capace di gestire flussi di dati di lunghezza arbitraria.

Durante la fase di cifratura ogni blocco di testo viene sottoposto a un'operazione XOR con il testo cifrato del blocco precedente prima di entrare nel primo round di AES; questo significa che l'input che subisce le trasformazioni di SubBytes, ShiftRows, MixColumns e AddRoundKey è già una miscela di dati attuali e storici della sessione.

Il punto più critico di questa integrazione emerge però durante la decifratura. In questa fase, il blocco cifrato C_j viene processato singolarmente dall'algoritmo AES inverso (utilizzando la IS-box e le chiavi di round in ordine decrescente); l'output di questa operazione non è ancora il messaggio originale, ma un valore intermedio. Per recuperare il testo in chiaro P_j il sistema deve eseguire un ultimo XOR tra questo valore intermedio e il blocco cifrato precedente C_{j-1} .

Questa struttura implica che AES, pur essendo matematicamente robusto, operi in "isolamento" all'interno del CBC. Esso non ha modo di verificare se il blocco che sta decifrando sia stato manomesso durante la trasmissione. Poiché lo XOR finale avviene con un dato (C_{j-1}) che è sotto il potenziale controllo di un attaccante, il sistema diventa intrinsecamente malleabile. Se un malintenzionato altera un byte nel blocco cifrato precedente, l'errore si propagherà esattamente nel byte corrispondente del testo decifrato dopo lo XOR.

È proprio questa interazione tra la decifratura AES e la struttura a catena del CBC che permette l'esistenza di vulnerabilità basate sulla manipolazione del testo, come il Padding Oracle Attack, qualora il sistema fornisca feedback sulla validità della struttura finale del messaggio.

3 Padding PKCS#7

I cifrari a blocchi, come AES, elaborano i dati in unità di dimensione fissa, chiamate appunto "blocchi". Per AES ogni blocco deve essere composto esattamente da 16 byte (128 bit). Tuttavia i messaggi che si trasmettono hanno lunghezze arbitrarie e raramente terminano coincidendo perfettamente con il confine dell'ultimo blocco.

Il padding è la tecnica utilizzata per risolvere questo problema di allineamento; consiste nell'aggiungere una serie di byte supplementari alla fine del testo in chiaro originale, fino a raggiungere la lunghezza richiesta dall'algoritmo di cifratura. Senza il padding l'algoritmo di cifratura non sarebbe in grado di processare i dati rimanenti, poiché le operazioni matematiche interne (come le trasformazioni di round di AES) richiedono una matrice di stato completa per funzionare.

L'uso del padding non consiste, però, nell'aggiunta di valori casuali; deve seguire regole standardizzate e deterministiche. Questo è fondamentale perché, una volta che il destinatario ha decifrato il messaggio, deve essere in grado di identificare dove finisce il testo reale e dove inizia il padding, così da poter rimuovere i byte extra e restituire il messaggio originale integro.

Se il padding non fosse strutturato correttamente, il destinatario rischierebbe di confondere parte del messaggio con il padding o di includere byte di errati nel messaggio finale decifrato.

3.1 Standard PKCS#7

Lo standard PKCS#7 (Public-Key Cryptography Standards #7) è il protocollo di padding più diffuso per i cifrari a blocchi. La sua popolarità nell'utilizzo con AES in modalità CBC deriva dalla sua implementazione semplice e dalla sua capacità di gestire in modo non ambiguo messaggi di qualsiasi lunghezza, garantendo che il destinatario possa sempre ricostruire il dato originale senza errori.

La regola principale dello standard PKCS#7 sancisce che il valore dei byte aggiunti per completare il blocco è uguale al numero di byte che vengono inseriti. Se la dimensione del blocco è B (per AES, $B = 16$ byte) e la lunghezza del messaggio è L , il numero di byte di padding N da aggiungere è calcolato come $N = B - (L \bmod B)$.

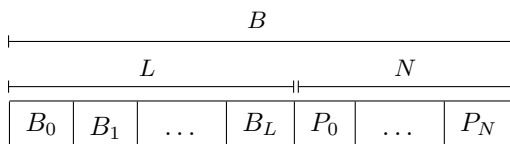
Il valore di ognuno degli N byte di riempimento sarà esattamente N . Ad esempio, se manca 1 byte si aggiunge $\{0x01\}$; se mancano 2 byte si aggiunge $\{0x02\}\{0x02\}$ e così via.

Una caratteristica fondamentale, che rende PKCS#7 robusto, è la gestione dei messaggi la cui lunghezza è già un multiplo esatto di 16. Lo standard impone l'aggiunta di un intero blocco supplementare di padding (16 byte con valore $\{0x10\}$). Questa operazione è necessaria perché, in fase di decifratura, il destinatario guarda sempre l'ultimo byte del messaggio decifrato per sapere quanti byte scartare.

Se il messaggio originale terminasse con il valore $\{0x01\}$ e non venisse aggiunto un blocco di padding, il sistema eliminerebbe erroneamente l'ultimo byte del messaggio reale riconoscendolo come padding. L'aggiunta obbligatoria del padding elimina ogni ambiguità.

L'uso di PKCS#7 nella modalità operativa CBC è diventato uno standard di fatto per diversi motivi:

- Verificabilità: permette al destinatario di effettuare un controllo di integrità sintattica. Se dopo la decifratura l'ultimo byte indica 0x03, ma i due byte precedenti non sono 0x03, il sistema sa immediatamente che il testo cifrato è stato manomesso o trasmesso in modo errato
- Efficienza: non richiede calcoli complessi; basta una semplice operazione di lettura e confronto dell'ultimo blocco
- Malleabilità controllata: nonostante la modalità CBC sia malleabile, lo standard PKCS#7 definisce una "grammatica" rigida



3.2 Padding in CBC

L'unione dello standard PKCS#7 con la modalità CBC e l'algoritmo AES crea un protocollo di decifratura a tre fasi che deve essere eseguito in un ordine rigoroso.

Quando un sistema riceve un testo cifrato e deve restituire il testo in chiaro esegue le operazioni in quest'ordine:

1. Decifratura AES: il blocco cifrato corrente C_j viene processato attraverso i round inversi di AES. Utilizzando la chiave segreta e la IS-box, il sistema "annulla" la cifratura AddRoundKey e le trasformazioni di round producendo l'Intermediate State. In questa fase il sistema non sa ancora nulla del testo in chiaro o del padding
2. Operazione XOR: il valore intermedio ottenuto viene messo in XOR con il blocco cifrato precedente C_{j-1} (o con l'IV se si tratta del primo blocco). Solo dopo questo passaggio si ottiene il blocco di testo in chiaro P_j
3. Validazione PKCS#7: una volta ottenuto P_j per l'ultimo blocco il sistema non lo consegna immediatamente all'utente; questo perchè prima deve verificare che il padding sia valido secondo lo standard PKCS#7. Il software analizza l'ultimo byte del messaggio, con il valore dei byte finali deve essere uguale al loro numero

Successivamente, se la validazione ha esito positivo, il padding viene rimosso e il messaggio è considerato integro. Se invece la sequenza di byte finali non rispetta le regole del PKCS#7 il sistema interrompe l'operazione e solleva un'eccezione.

4 Padding Oracle Attack

Il Padding Oracle Attack rappresenta una delle pietre miliari della crittoanalisi moderna. Presentato per la prima volta dal crittografo Serge Vaudenay nel 2002, questo attacco ha dimostrato come un protocollo crittografico possa essere completamente violato non colpendo la robustezza matematica del cifrario, ma sfruttando le informazioni trapelate dall'implementazione del sistema.

A differenza degli attacchi a forza bruta, che tentano di indovinare la chiave segreta attraverso miliardi di combinazioni, il Padding Oracle è un attacco di tipo side-channel. Ovvero, si basa sull'osservazione delle risposte di un server riguardo alla validità del padding di un testo cifrato inviato dall'attaccante. Questa tecnica permette di decifrare un intero messaggio intercettato inviando una serie di varianti modificate del testo cifrato e analizzando la reazione del sistema.

Nonostante siano passati più di vent'anni dalla sua scoperta, questo attacco rimane estremamente pericoloso e attuale per diversi motivi:

- Efficienza computazionale: mentre rompere AES richiederebbe tempi esponenzialmente alti, un Padding Oracle Attack può decifrare un blocco di 16 byte in una media di poche centinaia di richieste (massimo $16 \cdot 128 = 2048$), rendendolo eseguibile in pochi secondi da un normale computer
- Persistenza del CBC: la modalità Cipher Block Chaining è stata per decenni lo standard de facto nel protocollo TLS (alla base dell'HTTPS) e in molte librerie di cifratura legacy ancora in uso in sistemi bancari e industriali

Nel corso degli anni varianti di questo attacco hanno colpito standard massivi; per esempio si sono scoperte vulnerabilità in framework come ASP.NET e JavaServer Faces, dimostrando che anche implementazioni considerate sicure possono cadere vittima di sottili differenze nei messaggi d'errore o nei tempi di risposta del server.

L'impatto di tale vulnerabilità risiede nel fatto che trasforma un sistema progettato per la riservatezza in uno strumento che, involontariamente, aiuta l'attaccante a decifrare i propri dati; rendendo, così, la scelta della modalità operativa (CBC) e la gestione degli errori fattori tanto critici quanto la lunghezza della chiave stessa.

4.1 Oracolo

In crittografia, un oracolo è un'entità (solitamente un server o un'applicazione) che risponde a specifiche interrogazioni fornendo informazioni parziali, ma fondamentali, su un dato protetto. Nel caso del Padding Oracle Attack, l'oracolo non rivela mai direttamente la chiave segreta o il testo in chiaro, ma conferma o nega la validità della struttura di un messaggio decifrato.

Un oracolo agisce come una "scatola nera"; l'attaccante invia un testo cifrato modificato e riceve una risposta binaria. Sebbene il sistema crittografico AES sia matematicamente sicuro, la falla nasce dal feedback che il sistema restituisce durante la fase di decifratura. Un oracolo di padding esiste ogni volta che un'applicazione elabora un testo cifrato e, in caso di errore, permette all'attaccante di distinguere se il fallimento è dovuto a un problema di padding non valido o a un altro tipo di errore (come un errore di autenticazione o di logica applicativa).

La perdita di informazioni può avvenire attraverso diversi canali, definiti canali laterali (side-channels):

- Messaggi d'errore espliciti: è il caso più semplice. Il server restituisce un messaggio testuale come **Invalid Padding** o un codice di errore HTTP specifico (**500 Internal Server Error**) solo quando il padding PKCS#7 non è corretto; mentre restituisce un errore generico di **Login Failed** quando il padding è giusto ma le credenziali sono errate
- Differenze nei tempi di risposta (Timing Attack): anche se il server restituisce lo stesso messaggio di errore per ogni fallimento, la verifica del padding avviene solitamente prima di altre elaborazioni. Se il controllo del padding fallisce il sistema interrompe l'operazione immediatamente; se invece è corretto prosegue con la logica applicativa. Questa breve differenza nel tempo di esecuzione può essere misurata dall'attaccante per dedurre se il padding era valido o meno

L'attaccante sfrutta l'oracolo come una bussola; inviando migliaia di varianti di un blocco cifrato e osservando quando l'oracolo smette di dare l'errore di padding, può determinare con certezza matematica il valore dell'ultimo byte del testo decifrato.

Quindi, l'oracolo trasforma un problema di ricerca esaustiva (decifrare un intero blocco) in un problema di interrogazione iterativa byte per byte. Ogni volta che l'oracolo risponde **Padding OK**, l'attaccante ha appena ottenuto una parte del blocco che gli permetterà di calcolare il valore intermedio e, successivamente, il testo in chiaro originale.

4.2 Meccanismo dell'Attacco

L'obiettivo dell'attaccante non è indovinare la chiave AES; ma isolare il Valore Intermedio (I), ovvero l'output della decifratura AES prima che avvenga l'ultimo XOR del protocollo CBC.

L'attacco si basa sulla manipolazione del blocco di testo cifrato C_{j-1} per osservare gli effetti sul blocco di testo in chiaro P_j . Grazie alla malleabilità del CBC, l'attaccante può manipolare il testo in chiaro decifrato dal server semplicemente alterando i byte del blocco cifrato precedente che ha intercettato.

Come analizzato nel capitolo 2, la formula di decifratura per un blocco è: $P_j = D(K, C_j) \oplus C_{j-1}$. Si definisce il Valore Intermedio come:

$$I_j = D(K, C_j)$$

Di conseguenza si può riscrivere la formula di decifratura come:

$$P_j = I_j \oplus C_{j-1}$$

L'attaccante possiede C_j e C_{j-1} (intercettati ascoltando il traffico, per esempio con Man-in-the-Middle Attacck), ma non conosce né I_j né P_j . Tuttavia, poiché I_j è fisso (finché non cambia la chiave o il blocco cifrato), ogni modifica a un bit di C_{j-1} causerà una modifica identica nello stesso bit di P_j .

L'attaccante lavora a ritroso partendo dall'ultimo byte del blocco (16° byte). L'obiettivo è forzare il server a produrre un padding PKCS#7 valido.

L'attaccante prende il blocco C_{j-1} ed inizia a modificare l'ultimo byte ($C_{j-1}[16]$) provando tutti i 128 valori possibili (da $\{0x00\}$ a $\{0xFF\}$), inviando ogni variante al server (l'oracolo).

Per 127 tentativi il server risponderà **Invalid Padding**; ma per un valore specifico, lo XOR produrrà un testo in chiaro che termina con $\{0x01\}$. In quel momento, l'oracolo risponderà **Padding OK**. Quando l'attaccante riceve **Padding OK** sa con certezza che $P_j[16] = \{0x01\}$; conoscendo il valore del byte modificato che ha inviato ($C'_{j-1}[16]$), può calcolare il valore intermedio tramite XOR:

$$I_j[16] = P_j[16] \oplus C'_{j-1}[16] = \{0x01\} \oplus C'_{j-1}[16]$$

Una volta trovato il valore intermedio dell'ultimo byte, l'attaccante passa al penultimo (15° byte); per farlo, deve forzare un padding di valore $\{0x02\}\{0x02\}$.

Imposta il 16° byte del testo cifrato in modo che, con il Valore Intermedio già trovato, produca $\{0x02\}$; inizia a variare il 15° byte del testo cifrato finché l'oracolo non risponde nuovamente **Padding OK** (ovvero il blocco finisce correttamente in $\{0x02\}\{0x02\}$). Calcola poi: $I_j[15] = \{0x02\} \oplus C'_{j-1}[15]$.

Il processo prosegue iterativamente verso sinistra finché tutti i 16 byte del Valore Intermedio I_j non sono stati estratti.

Per la ricostruzione del testo in chiaro originale, l'attaccante non ha più bisogno dell'oracolo perchè ora che possiede l'intero blocco del Valore Intermedio (I_j). Può quindi scoprire il messaggio originale (P_j) semplicemente applicando lo XOR con il testo cifrato originale intercettato all'inizio (C_{j-1}):

$$P_j = I_j \oplus C_{j-1}$$

4.3 Logica dell'Attacco

L'attaccante vuole che l'oracolo restituisca **Padding OK**, ed il modo più semplice per ottenere un questo messaggio dal server è far sì che l'ultimo blocco decifrato termini con almeno un byte di padding valido. Quindi il caso più facile da creare è un blocco che finisce con $\{0x01\}$.

L'attaccante non può toccare la decifratura AES (che richiede la chiave), ma controlla totalmente C'_{j-1} (il blocco cifrato precedente modificato). La formula che avviene dentro il server è:

$$P_j[16] = I_j[16] \oplus C'_{j-1}[16]$$

$I_j[16]$ è un valore fisso ma sconosciuto, perchè è l'uscita di AES; l'attaccante cambia $C'_{j-1}[16]$ provando tutti i valori da 0 a 127.

Statisticamente, esisterà un solo valore di $C'_{j-1}[16]$ che, messo in XOR con il valore sconosciuto $I_j[16]$, darà come risultato esattamente $\{0x01\}$.

Quando l'attaccante indovina quel valore ($C'_{j-1}[16]$): il server esegue la decifratura, vede che l'ultimo byte è $\{0x01\}$, il software di validazione controlla il padding e conferma se è valido, l'oracolo risponde: **Padding OK**.

In breve l'attaccante non sa cosa c'è dentro il server, ma usa il valore $\{0x01\}$ come una "chiave di lettura". Nel momento in cui il server non dà errore, l'attaccante ha la prova matematica che lo XOR ha prodotto $\{0x01\}$. A quel punto, conoscendo il risultato dello XOR ($\{0x01\}$) e il valore che ha inviato lui ($C'_{j-1}[16]$), può ricavare il valore intermedio con la formula inversa:

$$I_j[16] = C'_{j-1}[16] \oplus \{0x01\}$$

Se l'attaccante cambiasse i byte a caso potrebbe capitare che il server risponda **Padding OK** perché il blocco potrebbe terminare con un padding corretto; ma questo sarebbe un problema per l'attaccante, non un vantaggio.

Uno dei principali problemi è il "Rumore" del padding multi-byte. Nell'ipotesi in cui l'attaccante, per esempio, trovasse un padding di $\{0x02\} \{0x02\}$ per puro caso, non saprebbe quale dei due byte ha causato il successo. Dato che l'obiettivo dell'attaccante è capire con precisione quale byte ha causato la modifica, è fondamentale che il primo padding valido che trova sia univocamente identificabile. 2. Perché si "punta" sempre allo $0x01$ all'inizio?

Come già spiegato precedentemente l'attaccante esegue una strategia precisa per eliminare l'ambiguità: all'inizio forza il padding di un solo byte ($\{0x01\}$). Questo approccio ha diversi vantaggi, tra cui; l'isolamento, infatti modifica solo l'ultimo byte ($C'_{j-1}[16]$) e lascia invariati i primi 15 byte del blocco; statisticamente la probabilità che l'ultimo byte decifrato diventi $\{0x01\}$ è di 1 su 128. Ed inoltre la probabilità che gli ultimi due byte diventino casualmente $\{0x02\} \{0x02\}$ è 1 su 16.384.

Una volta che l'attaccante ha trovato il valore intermedio dell'ultimo byte ($I_j[16]$), costringe il server a cercare il padding successivo. Per trovare il 15° byte l'attaccante imposta il 16° byte del testo cifrato in modo che il risultato della decodifica sia $\{0x02\}$; ora può farlo perché conosce $I_j[16]$. Eseguendo l'operazione:

$$C'_{j-1}[16] = I_j[16] \oplus \{0x02\}$$

Successivamente inizia a variare il 15° byte finché l'oracolo non restituisce nuovamente **Padding OK**; in quel momento l'attaccante sa che il padding è $\{0x02\} \{0x02\}$.

La logica è la stessa per i byte successivi; una volta trovato il valore intermedio di un byte l'attaccante lo usa per forzare il padding del byte precedente, così via fino a ricostruire l'intero blocco di testo in chiaro.

L'obiettivo dell'attaccante è decidere quale padding il server deve trovare ($\{0x01\}$, poi $\{0x02\} \{0x02\}$, poi $\{0x03\} \{0x03\} \{0x03\} \dots$). L'oracolo non gli fornisce informazioni generiche, ma dà una conferma se il byte che ha modificato l'attaccante ha fatto sì che il padding diventasse corretto. Se il server risponde **Padding OK**, l'attaccante ha ottenuto un byte di informazione.

Non punta a ricostruire l'intero blocco di testo in chiaro tutto in una volta, ma a recuperare parti del blocco ed a ricostruirlo byte per byte sfruttando le risposte del server.

4.4 Esempio Logica dell'Attacco

L'attaccante ha intercettato i blocchi cifrati C_{j-1} e C_j ; il primo passo è capire quale valore del byte modificato ($C'_{j-1}[16]$) produce un messaggio **Padding OK** di $\{0x01\}$.

Una volta scoperto il valore che valida il padding posso calcolare il 16° byte del Valore Intermedio:

$$I_j[16] = C'_{j-1}[16] \oplus \{0x01\}$$

Ora deve recuperare il 15° byte di I_j ; per farlo; deve perciò forzare un padding di $\{0x02\} \{0x02\}$. Dato che conosce già $I_j[16]$, può impostare il 16° byte del testo cifrato modificato $C'_{j-1}[16]$ in questo modo:

$$C'_{j-1}[16] = I_j[16] \oplus \{0x02\}$$

Successivamente è necessario calcolare il 15° byte del testo cifrato modificato $C'_{j-1}[15]$ finché l'oracolo non restituisce **Padding OK**. Una volta trovato calcolo il 15° byte del Valore Intermedio:

$$I_j[15] = C'_{j-1}[15] \oplus \{0x02\}$$

Conoscendo $I_j[15]$ e $I_j[16]$ posso forzare un padding di $\{0x03\} \{0x03\} \{0x03\}$ per trovare il 14° byte del Valore Intermedio. Prima calcola il 16° ed il 15° byte del testo cifrato modificato:

$$C'_{j-1}[16] = I_j[16] \oplus \{0x03\}$$

$$C'_{j-1}[15] = I_j[15] \oplus \{0x03\}$$

Successivamente è necessario calcolare il 14° byte del testo cifrato modificato $C'_{j-1}[14]$ finché l'oracolo non restituisce **Padding OK**. Una volta trovato calcoliamo il 14° byte del Valore Intermedio:

$$I_j[14] = C'_{j-1}[14] \oplus \{0x03\}$$

Continuando in questo modo, l'attaccante riesce a ricostruire l'intero blocco del Valore Intermedio I_j . Una volta ottenuto I_j , può ricostruire il testo in chiaro originale P_j semplicemente facendo lo XOR con il blocco cifrato originale C_{j-1} :

$$P_j = I_j \oplus C_{j-1}$$

4.5 Contromisure

L'attacco Padding Oracle si basa sulla capacità dell'attaccante di modificare il testo cifrato e ricevere un feedback (l'oracolo). Le difese devono quindi mirare a eliminare questo feedback o a rendere impossibile la manipolazione del testo cifrato.

La difesa più efficace consiste nell'utilizzare schemi di cifratura che garantiscano non solo la riservatezza, ma anche l'integrità e l'autenticità. Invece di usare la modalità CBC standard si utilizzano modalità come GCM (Galois/Counter Mode) o CCM, perché questi algoritmi aggiungono un Tag di Autenticazione (MAC) al messaggio. La logica tecnica si basa sul fatto che prima di tentare la decifratura e il controllo del padding, il server verifica il Tag; se l'attaccante modifica anche un solo bit di C_{j-1} il Tag risulterà invalido. Di conseguenza il server scarta il pacchetto immediatamente, senza nemmeno guardare il padding.

Un'altra possibile soluzione consiste nell'implementazione del principio "Encrypt-then-MAC" (EtM). Permette di continuare ad usare la modalità operativa CBC, ma con un codice di autenticazione del messaggio (MAC) esterno; ed è l'unico veramente sicuro contro i Padding Oracle.

Il procedimento è molto semplice: si cifra il messaggio:

$$C = E(K, P)$$

Si calcola il MAC sul testo cifrato:

$$T = MAC(K, C)$$

Il destinatario verifica prima T; se il testo cifrato è stato manipolato dall'attaccante la verifica fallisce ed il processo si ferma.

Una difesa parziale, ma spesso insufficiente se usata da sola, riguarda il modo in cui il server risponde agli errori. Il server non dovrebbe mai specificare se l'errore è dovuto al padding o a qualcos'altro; tuttavia questa difesa è debole a causa dei Side-Channel Attacks basati sul tempo (Timing Attacks). Anche se il server restituisce un errore generico "Errore di decifratura", il tempo impiegato per rispondere potrebbe variare. La soluzione consiste nell'implementare risposte in tempo costante, rendendo le risposte del server identiche sia nei contenuti che nei tempi.

Un'ultima possibile soluzione è l'eliminazione del Padding usando la modalità CTR. La modalità CTR (Counter Mode) trasforma il cifrario a blocchi in un cifrario a flusso; quindi, poiché il messaggio viene cifrato bit a bit tramite XOR con un keystream, non c'è bisogno di riempire l'ultimo blocco. Si ottiene così un sistema di cifratura senza padding, in cui non esiste alcun "Padding Oracle" da interrogare.